Texto

##

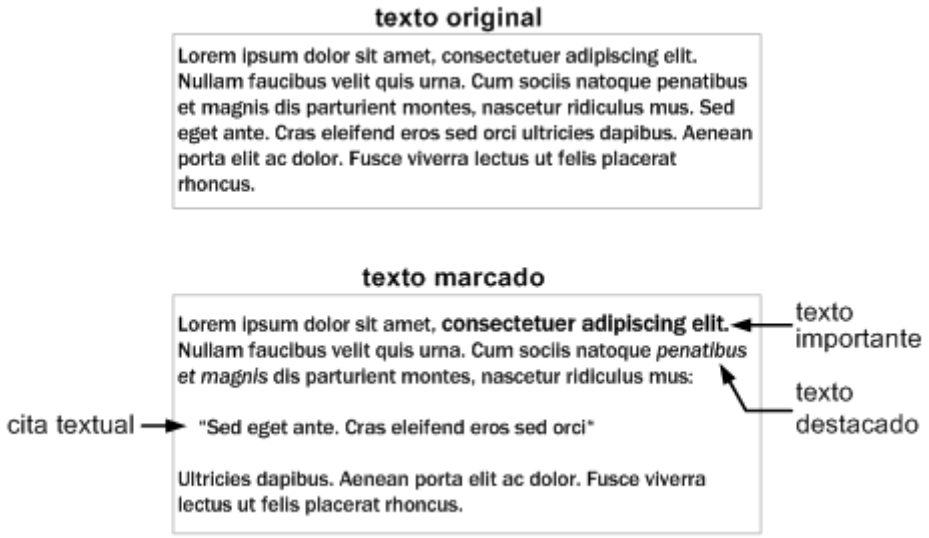
Intro

La mayor parte del contenido de las páginas HTML habituales está formado por texto, llegando a ser más del 90% del código de la página. Por este motivo, es muy importante conocer los elementos y etiquetas que define HTML para el manejo del texto.

El lenguaje HTML incorpora al tratamiento del texto muchas de las ideas y normas establecidas en otros entornos de publicación de contenidos. De esta forma, HTML define etiquetas para estructurar el contenido en secciones, artículos y párrafos y define otras etiquetas para marcar elementos importantes dentro del texto.

El proceso de estructurar un texto simple consiste en indicar las diferentes secciones y artículos que componen el texto, y dentro de ellos, las subsecciones y párrafos que lo componen, para lo que utilizaremos distintas etiquetas.

Una vez definida la estructura básica de los contenidos de la página, el siguiente paso consiste en marcar los diferentes elementos dentro del propio texto: definiciones, abreviaturas, textos importantes, textos modificados, citas a otras referencias, etc, para lo que usaremos otro conjunto de etiquetas.



Encabezados de sección

Los encabezados de sección (*headings* en inglés) se utilizan para establecer un texto que actúa como título de dicha sección. Normalmente se insertan dentro de una etiqueta `<section>`, aunque no es estrictamente necesario.

HTML define seis niveles jerárquicos de encabezados de sección, mediante las etiquetas `<h1>`, `<h2>`, `<h3>`, `<h4>`, `<h5>` y `<h6>`, que son elementos en bloque, donde `<h1>` es la de mayor nivel jerárquico. Debe quedar claro que `<h1>`, `<h2>`, etc. NO indican el orden en el que aparecen, sino el nivel de importancia (jerarquía) del título.

Es importante resaltar también que las etiquetas `<h1>`, ..., `<h6>` definen títulos de sección, no secciones completas. Por este motivo, no es necesario encerrar los contenidos de una sección dentro de ellas, sino solamente el título.

Veamos un ejemplo:

```
<h1>Mi página web</h1>
<section>
  <h2>Novedades</h2>
  <article>
  <h3>La informática mola</h3>
    ...
  </article>
  <article>
    <h3>HTML es guay</h3>
    <section>
      ...
    </section>
  </article>
</section>
<section>
  <h2>Para saber más</h2>
  <!-- ... -->
</section>
```

Párrafos

Una de las etiquetas más utilizadas de HTML es la etiqueta `<p>` (del inglés *paragraph*), que permite definir los párrafos que forman el texto de una página. Para delimitar el texto de un párrafo, se encierra ese texto con la etiqueta `<p>párrafo</p>`.
Los párrafos creados con HTML son elementos de bloque, por lo que por defecto ocupan toda la anchura de la ventana del navegador.

Marcado básico de texto

Una vez estructurado el texto en párrafos y secciones, el siguiente paso es el marcado de los elementos que componen el texto. Los textos habituales están formados por elementos como palabras en negrita o cursiva, anotaciones y correcciones, citas a otros documentos externos, etc. HTML proporciona varias etiquetas para marcar cada uno de los diferentes tipos de texto:

- `<em>`: Sirve para enfatizar (resaltar) un texto (del inglés *emphasized*). Es un elemento en línea. Por defecto el navegador lo muestra en cursiva.
- `<strong>`: Sirve para indicar que un texto es muy importante (más que enfatizado, fuerte). Es un elemento en línea. Por defecto el navegador lo muestra en negrita.
- `<mark>`: Representa un texto resaltado (resaltado) debido a su relevancia en un contexto particular, por ejemplo las apariciones de un texto buscado. Los navegadores por defecto lo muestran con un fondo en amarillo intenso.
- `<abbr>`: Indica que un determinado texto corresponde a una abreviatura o un acrónimo (siglas). Deberemos usar el atributo `title` para proporcionar el significado de la abreviatura o del acrónimo. Es un elemento en línea. La mayoría de los navegadores muestran por defecto el texto subrayado con una línea de puntos.
- `<dfn>`: Sirve para proporcionar la definición del término encerrado por la etiqueta (del inglés *definition*). Se usa su atributo `title` para establecer la definción. Es un elemento en en línea.
- `<q>`: Sirve para indicar que un determinado texto es una cita (algo que ha dicho o escrito otra persona, del inglés *quote*). Gracias a su atributo `cite` podemos especificar la URL dónde encontrar la fuente de la cita. Es un elemento en línea.
- `<cite>`: Representa una referencia a un trabajo creativo, como el título de un libro, de un disco, un cuadro, etc. Es un elemento en línea.
- `<time>`: Indica que un determinado texto hace referencia a una fecha y/o una hora. En el caso de que el texto no corresponda explícitamente a una fecha y/o una hora, como por ejemplo "día de San Valentín", usaremos el atributo `datetime` para indicar la fecha y/o la hora real a la que hace referencia. Es un elemento en línea.
- `<del>`: Sirve para indicar que un determinado texto ya no es válido o no está actualizado (del inglés *deleted*). Por defecto el navegador muestra el texto tachado. El atributo específico `cite` recibe como valor la URL donde se explica el motivo por el que el texto ya no es válido, y con el atributo `datetime` se indica la fecha en la que el texto ha dejado de ser válido.
- `<ins>`: Indica que un determinado texto ha sido añadido posteriormente al documento (del inglés *inserted*). Se suele usar en conjunción con la etiqueta `<del>` para indicar el nuevo texto que sustituye al que ya no es válido. Posee los mismos atributos específicos que dicha etiqueta.

- `<blockquote>`: Sirve para indicar que un determinado texto (párrafo) es una cita (algo que ha dicho o escrito otra persona, en español bloque de cita). Gracias a su atributo `cite` podemos especificar la URL dónde encontrar la fuente de la cita. Se diferencia de la etiqueta `<q>` en que `<blockquote>` es un elemento en bloque y se usa para citas mostradas en su propio "párrafo". Los navegadores por defecto aplican un sangrado adicional al elemento.
- `<detail>` y `<summary>`: El elemento `<detail>` representa un bloque de texto que por defecto aparece contraído pero que puede ser expandido (detalle). Como primer hijo de la etiqueta `<detail>` se usa una etiqueta `<summary>` para indicar el texto que debe mostrarse cuando el bloque esté contraído (en español, resumen). Por defecto aparecerá contraído. Si queremos que nuestro bloque de texto aparezca inicialmente expandido usaremos el atributo `open="open"` en la etiqueta `<detail>`.

También tenemos disponibles una serie de etiquetas a usar cuando el texto corresponde a código de programación o a fórmulas matemáticas:

- `<var>`: El texto corresponde a una variable de una fórmula o código de programación.
- `<sub>`: Indica que el texto es un subíndice.
- `<sup>`: El texto es un superíndice.
- `<kdb>`: Indica que el texto corresponde a una tecla o combinación de teclas.
- `<samp>`: El texto corresponde a un mensaje de salida de un programa.
- `<code>`: El texto corresponde a código de programación.

Veamos un ejemplo de uso de varias de estas etiquetas:

See the Pen [Marcado básico de texto](#) by Pedro Joya (@pedrojoya) on CodePen.

```
<p>Lorem <em>ipsum</em> dolor <strong>sit amet</strong>, consectetur
<mark>adipisicing elit</mark>. Inventore veniam magnam perspiciatis, <abbr
title="Hyper Text Manipulation Language">HTML</abbr>, <abbr title="Linux Apache
MySQL PHP">LAMP</abbr>, sit ea natus voluptates eveniet, amet ab voluptatem <dfn
title="Parecido">alias</dfn> aliquid <q cite="http://www.iessaladillo.es">cumque
mollitia</q> corporis praesentium voluptatibus incidunt dolor? El <time
datetime="2016-12-25">día de navidad</time></p>
<p><del>Esto es antiguo</del><ins>Esto es nuevo</ins></p>
<blockquote cite="https://www.letras.com/kiko-veneno/traspaso/">Yo pa que quiero
un trabajo fijo si contigo tengo ya bastante</blockquote>
<details open="open">
  <summary>Leer más</summary>
  <p>Lorem ipsum dolor sit amet, consectetur adipisicing elit. Commodi quos
ratione itaque facilis aspernatur quia quas, iure quisquam nobis, deleniti
voluptates sint asperiores debitis quaerat harum iste. Nam, magni, quam!</p>
  <p>Lorem ipsum dolor sit amet, consectetur adipisicing elit. Veritatis nulla
neque, at quia facere debitis ut odio atque tempora accusamus, id nam dolore
maiores iure! Vero nisi unde possimus atque.</p>
</details>
<p><var>x<sup>2</sup></var> + <var>y<sub>1</sub></var>=5</p>
<p>Para terminar pulse <kbd>Enter</kbd> y <kbd>Ctrl</kbd></p>
<p>Si ejecutamos el siguiente código de programación <code>print "No se ha
encontrado la solución"</code> la salida por pantalla es <samp>No se ha
encontrado solución</samp></p>
```

```
abbr {
  text-decoration: underline;
  -webkit-text-decoration-style: dotted;
}
kbd {
  font-family: "Helvetica Neue", Helvetica, Arial, sans-serif;
  color: #ffffff;
  padding: .3em;
  border: 1px solid rgba(0, 0, 0, 0.05);
  border-radius: 3px;
  background-clip: padding-box;
  background: #5e5e5e;
  box-shadow: 0 2px 2px rgba(0, 0, 0, 0.3), 0 1px 0 #444444, inset 0 1px 0
#868686;
}
details summary {
  display: inline;
  background: #D6A96C;
  border: 1px solid #825417;
  color: #fff;
  padding: .5em 1em;
  border-radius: 5px;
}
code {
  background: lightgrey;
}
```

> Demo de marcado básico de texto

Microdatos

Los microdatos son una especificación HTML que se emplea para anidar metadatos en el contenido existente de las páginas web, de manera que buscadores, *bots* y navegadores pueden extraer y procesar los microdatos a partir de una página y utilizarlos para proveer una experiencia más enriquecida para los usuarios. Los buscadores se benefician considerablemente del acceso directo a estos datos estructurados, ya que les permite entender la información contenida en las páginas y ofrecer mejores resultados a sus usuarios.

Los microdatos utilizan una serie de atributos en las etiquetas para describir tanto los elementos de datos como pares nombre-valor y así establecer propiedades de los elementos de datos. Asi, dispondremos de atributos como `itemscope` para indicar un elemento de datos, `itemprop` para indicar una de sus propiedades o `itemtype` para indicar su tipo.

En definitiva, los microdatos representan un intento de brindar una manera más sencilla de anotar elementos HTML con etiquetas legibles por máquinas. Veamos un ejemplo:

```
<div itemscope itemtype="http://schema.org/SoftwareApplication">
  <span itemprop="name">Angry Birds</span> -
  REQUIRES <span itemprop="operatingSystem">ANDROID</span><br>
  <link itemprop="applicationCategory" href="http://schema.org/GameApplication"/>
  <div itemprop="aggregateRating" itemscope
itemtype="http://schema.org/AggregateRating">
    RATING:
    <span itemprop="ratingValue">4.6</span> (
    <span itemprop="ratingCount">8864</span> ratings )
```

```
  </div>
  <div itemprop="offers" itemscope itemtype="http://schema.org/Offer">
    Price: $<span itemprop="price">1.00</span>
    <meta itemprop="priceCurrency" content="USD" />
  </div>
</div>
```

Propiedades CSS relacionadas con el tipo de letra

font-family

La propiedad básica que define CSS relacionada con la tipografía se denomina font-family y se utiliza para indicar el tipo de letra con el que se muestra el texto.

Propiedad	font-family
Valores	((nombre_familia | familia_generica) (,nombre_familia | familia_generica)*) | inherit
Se aplica a	Todos los elementos
Valor inicial	Depende del navegador
Heredada	Sí
Descripción	Establece el tipo de letra utilizado para el texto

El tipo de letra del texto se indica mediantes una lista de fuentes separadas por coma, ordenadas por orden de preferencia, donde cada elemento de la lista puede ser:

- El nombre de una familia tipográfica, como por ejemplo "Arial", "Verdana", "Garamond", etc.
- El nombre genérico de una familia tipográfica segura para la web (*web-safe fonts*), que no se refieren a ninguna fuente en concreto, sino que informan al navegador que busque una tipografía instalada en el sistema que pertenezca a dicha familia. Se utiliza normalmente con último elemento de la lista de fuentes, para que use como último caso.

Las familias genéricas seguras para la web son:

- `serif`: Tipografía con "serifa". Por ejemplo `Times New Roman`.
- `sans-serif`: Tipografía sin "serifa" (también llamada palo seco). Por ejemplo `Arial`.
- `cursive`: Tipografía cursiva. Por ejemplo `Comic Sans`.
- `fantasy`: Tipografía decorativa. Por ejemplo `Impact`.
- `monospace`: Tipografía monoespaciada (todas los caracteres tienen el mismo ancho). Por ejemplo `Courier New`.
  Los navegadores muestran el texto de las páginas web utilizando los tipos de letra instalados en el ordenador o dispositivo del propio usuario. De esta forma, si el diseñador indica en la propiedad `font-family` que el texto debe mostrarse con un tipo de letra especialmente raro o rebuscado, casi ningún usuario dispondrá de ese tipo de letra.

Para evitar el problema común de que el usuario no tenga instalada la fuente que quiere utilizar el diseñador, CSS permite indicar en la propiedad `font-family` más de un tipo de letra. El navegador probará en primer lugar con el primer tipo de letra indicado. Si el usuario la tiene instalada, el texto se muestra con ese tipo de letra. Si el usuario no dispone del primer tipo de letra indicado, el navegador irá probando con el resto de tipos de letra hasta que encuentre alguna fuente que esté instalada en el ordenador del usuario.

Evidentemente, el diseñador no puede indicar para cada propiedad `font-family` tantos tipos de letra como posibles fuentes parecidas existan. Para solucionar este problema se utilizan las familias tipográficas genéricas. Cuando la propiedad `font-family` toma un valor igual a `sans-serif`, el diseñador no indica al navegador que debe utilizar la fuente Arial, sino que debe utilizar "la fuente que más se parezca a Arial de todas las que tiene instaladas el usuario".

Por todo ello, el valor de `font-family` suele definirse como una lista de tipos de letra alternativos separados por comas. El último valor de la lista es el nombre de la familia tipográfica genérica que más se parece al tipo de letra que se quiere utilizar.

Veamos unos ejemplos:

```
/* Si el nombre de la fuente contiene espacios en blanco,
   debe encerrarse entre comillas */
body { font-family: Arial, Helvetica, sans-serif; }
body { font-family: "Times New Roman", Times, serif; }
body { font-family: "Courier New", Courier, monospace; }
body { font-family: Georgia, "Times New Roman", Times, serif; }
body { font-family: Verdana, Arial, Helvetica, sans-serif; }
```

Ya que las fuentes que se utilizan en la página deben estar instaladas en el ordenador del usuario, cuando se quiere disponer de un diseño complejo con fuentes muy especiales, se debe recurrir a soluciones alternativas. Una de las más usadas es utilizar Google Fonts, que corresponde a un repositorio de tipos de letras, en el que una vez seleccionada la fuente y la variante deseada se nos muestra información sobre los pasos necesarios para usarla en nuestra página.

Por ejemplo, si hemos seleccionado la fuente 'Lato' en su variante 'regular 400' en nuestro archivo HTML tendremos que introducir la siguiente etiqueta `<link>` dentro de `<head>`, y antes de la etiqueta `<link>` correspondiente a nuestro propio archivo CSS, para que sea importada cuando se muestre nuestra página:

```
<link href="https://fonts.googleapis.com/css?family=Lato" rel="stylesheet">
```

A partir de ese momento podremos hacer uso de dicha fuente en nuestro CSS de la misma forma que el resto de fuentes:

```
body { font-family: 'Lato', sans-serif; }
```

Veamos un ejemplo gráfico:

See the Pen Propiedad font-family by Pedro Joya (@pedrojoya) on CodePen.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8" />
  <link href="https://fonts.googleapis.com/css?family=Monoton|Noto+Sans"
rel="stylesheet">
  <title>Propiedad font-family</title>
</head>
<body>
  <p>Texto con fuente por defecto</p>
  <p class="monoton">Texto con la fuente Monoton</p>
  <p class="noto">Texto con la fuente Noto Sans</p>
</body>
</html>
```

```
p.monoton {
  font-family: 'Monoton', cursive;
}
p.noto {
  font-family: 'Noto Sans', sans-serif;
}
```

> Demo de uso de una fuente de Google Fonts

font-size

La propiedad `font-size` nos permite establecer el tamaño de la letra.

Propiedad	font-size
Valores	cte_tamaño_absoluto | cte_tamaño_relativo | unidad de medida | porcentaje | inherit
Se aplica a	Todos los elementos
Valor inicial	medium
Heredada	Sí
Descripción	Establece el tamaño de letra utilizado para el texto

CSS permite utilizar una serie de constantes para indicar el tamaño en esta propiedad:

- De tamaño absoluto: `xx-small`, `x-small`, `small`, `medium`, `large`, `x-large`, `xx-large`.
- De tamaño relativo: Toman como referencia el tamaño de letra del padre: `larger`, `smaller`. Veamos un ejemplo:

```
body { font-size: 1.2em; }
```

Por defecto, los navegadores establecen `16px` como tamaño de fuente base y asignan los siguientes tamaños a los títulos de sección: `<h1>`: `xx-large`, `<h2>`: `x-large`, `<h3>`: `large`, `<h4>`: `medium`, `<h5>`: `small` y `<h6>`: `xx-small`.

Tradicionalmente se ha indicado el tamaño del texto en la unidad `px` en el caso de páginas web y en la unidad `pt` (puntos) en el caso de documentos diseñados para ser impresos.

También podemos usar unidades relativas, como `em` o en porcentaje (%). En ambos casos el tamaño se calcula respecto al tamaño de fuente del elemento padre de al que se le está aplicando.

Una mejor alternativa es definir el tamaño de fuente del elemento raíz `<html>` y definir el tamaño fuente del resto de elemento respecto a éste, usando para ello la unidad `rem`. En el siguiente ejemplo se define el tamaño base a 10px, porque después nos va a ser muy fácil calcular mentalmente el resto de tamaños en `rem`. Por ejemplo:

```
html {
 font-size: 10px;
}
h1 {
 font-size: 3rem;
}
```

En el ejemplo anterior los `<h1>` se mostrarán con un tamaño de `30px`. Esta alternativa es mejor porque si después queremos cambiar el tamaño de todos los textos tan sólo tendremos que cambiar el tamaño de fuente del elemento `<html>`, ya que el resto es calculado relativo a éste. Por ejemplo, si queremos que en dispositivos grandes el texto se vea más grande, tan sólo tendremos que incluir una *media query* que modifique el tamaño de fuente de la etiqueta `<html>` para estos dispositivos.

Sin embargo, el ejemplo anterior tiene un pequeño inconveniente: el tamaño de fuente no se verá modificado si el usuario cambia las preferencias de tamaño de texto en el navegador, ya que hemos establecido a una cantidad fija de píxeles el tamaño base. Por este motivo, se suele establecer el tamaño de fuente base del elemento `<html>` al valor `62.5%`, proveniente de dividir el tamaño base deseado, `10px`, entre el tamaño base asignado por los navegadores `16px` y multiplicarlo por `100`, esto es `100 * 10px/16px = 62.5`. De esta manera los cálculos para el resto de elemento seguirán haciéndose en base a `10px`, pero cuando el usuario cambie el tamaño base en el navegador, el tamaño de fuente de la etiqueta `<html>` se verán afectado y como consecuencia el del resto de elementos. Así el ejemplo anterior quedaría como:

```
html {
 font-size: 62.5%;
}
h1 {
 font-size: 3rem;
}
```

Otra alternativa es la conocida como tipografía fluida, consistente en usar alguna unidad relativa al *viewport*, como `vw`, para definir el tamaño de fuente del elemento `html` y definir el tamaño de fuente del resto de elementos usando `rem`. De esta manera, el tamaño de letra de todos los elementos dependerán del tamaño de la anchura del *viewport* del dispositivo. El ejemplo anterior quedaría de la siguiente manera:

```
html {
 font-size: 2vw;
}
h1 {
 font-size: 3rem;
}
```

Debemos fijarnos en que NO existen (aún) las propiedades `min-font-size` y `max-font-size` por lo que trabajar con tamaños de fuentes especificadas en `vh` o `vw` puede hacer que la letra quede demasiado pequeña o demasiado grande en determinados tamaños de pantalla. En ese caso deberemos usar *media queries* para cambiar el tamaño base de la etiqueta `<html>` en determinados tamaños de *viewport* de dispositivo. Para saber sobre este aspecto se recomienda leer el siguiente artículo sobre tipografía fluida.

font-weight

La propiedad `font-weight` nos permite especificar el grosor de la letra.

Propiedad	**font-weight**
Valores	normal | bold | bolder | lighter | 100 | 200 | 300 | 400 | 500 | 600 | 700 | 800 | 900 | inherit
Se aplica a	Todos los elementos
Valor inicial	normal
Heredada	Sí
Descripción	Establece la anchura de la letra utilizada para el texto

Veamos un ejemplo:

```
h1 { font-weight: bold; }
```

Los valores que normalmente se utilizan son `normal` (el valor por defecto) y `bold` para los textos en negrita. El valor normal equivale al valor numérico 400 y el valor bold al valor numérico 700.

font-style

La propiedad `font-style` nos permite especificar el estilo de la letra.

Propiedad	**font-style**
Valores	normal | italic | oblique | inherit
Se aplica a	Todos los elementos
Valor inicial	normal
Heredada	Sí
Descripción	Establece el estilo de la letra utilizada para el texto

Veamos un ejemplo:

```
h2 { font-style: italic; }
```

Normalmente la propiedad `font-style` se emplea para mostrar un texto en cursiva mediante el valor `italic`. El valor `oblique` hace que la letra quede aún más tumbada que con `italic`.

font-variant

La propiedad `font-variant` permite establecer que la letra se muestre con la variante de letra versal (mayúsculas pequeñas).

Propiedad	font-variant
Valores	normal | small-caps | inherit
Se aplica a	Todos los elementos
Valor inicial	normal
Heredada	Sí
Descripción	Establece el estilo alternativo de la letra utilizada para el texto

Veamos un ejemplo:

```
h3 { font-variant: small-caps; }
```

La propiedad `font-variant` no se suele emplear habitualmente.

font-stretch

Si hemos establecido como tipo de letra una fuente que tenga varios estilos, en inglés *faces* (delgada, normal, condensada, gruesa, etc.) podemos usar la propiedad `font-stretch` para indicar si la letra debe aparecer expandida, condensada, etc. Atendiendo al valor especificado el navegador usará el mejor estilo disponible.

Propiedad	**font-stretch**
Valores	normal
Se aplica a	normal | wider | narrower | ultra-condensed | extra-condensed | condensed | semi-condensed | semi-expanded | expanded | extra-expanded | ultra-expanded | inherit
Valor inicial	normal
Heredada	Sí
Descripción	Establece el estilo (*face*) de la letra utilizada para el texto

Veamos un ejemplo:

```
p { font-stretch: ultra-expanded; }
```

font-size-adjust

La propiedad `font-size-adjust` permite ajustar la altura (*x-height*) de la letra para hacerla más legible, ya que para un determinado tamaño de letra, algunas fuentes son más difíciles de leer que otras. Esto es especialmente importante si el tipo de letra no está disponible y se usa el siguiente tipo de letra especificado en `font-family` y éste tiene un ratio de aspecto diferente que hace que se lea peor.

Propiedad	**font-size-adjust**
Valores	none | <número>
Se aplica a	Todos los elementos
Valor inicial	none
Heredada	Sí
Descripción	Ajusta la proporción de altura de la fuente

Veamos un ejemplo:

```
p {
  font: 12px Verdana, Helvetica, sans-serif;
  font-size-adjust: 0.58;
}
```

Esta propiedad no es soportada aún por la mayoría de los navegadores actuales.

font

CSS proporciona una propiedad tipo atajo (*shorthand*) denominada `font` que permite indicar de una sola vez los valores de algunas de las propiedades relacionadas con la letra.

Propiedad	font
Valores	((font-style | font-variant | font-weight)? font-size (/ line-height)? font-family) | caption | icon | menu | message-box | small-caption | status-bar | inherit
Se aplica a	Todos los elementos
Valor inicial	-
Heredada	Sí
Descripción	Establece de una vez los valores de algunas de las propiedades para el tipo de letra

El orden en el que se deben indicar las propiedades del texto es el siguiente:

- En primer lugar y de forma opcional se indican el `font-style`, `font-variant` y `font-weight` en cualquier orden.
- A continuación, se indica obligatoriamente el valor de `font-size` seguido opcionalmente por el valor de `line-height` (no lo hemos visto aún).
- Por último, se indica obligatoriamente el `font-family` a utilizar.

Veamos un ejemplo:

See the Pen [Propiedad font](#) by Pedro Joya (@pedrojoya) on CodePen.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8" />
  <link href="https://fonts.googleapis.com/css?family=Monoton|Noto+Sans"
rel="stylesheet">
  <title>Propiedad font-family</title>
</head>
<body>
  <p>Texto con fuente por defecto</p>
  <p class="monoton">Texto con la fuente Monoton</p>
  <p class="noto">Texto con la fuente Noto Sans</p>
</body>
</html>
```

```
p.monoton {
  font: italic small-caps 2em 'Monoton', cursive;
}
p.noto {
  font: bolder 1.5em 'Noto Sans', sans-serif;
}
```

> Demo de la propiedad font

Propiedades CSS relacionadas con el texto

color

El atributo `color` se usa para establecer el color de la letra.

Propiedad	**color**
Valores	color | inherit
Se aplica a	Todos los elementos
Valor inicial	Depende del navegador
Heredada	Sí
Descripción	Establece el color de letra utilizado para el texto

Veamos algunos ejemplos:

```
p { color: black; }
p { color: #B1251E; }
p { color: rgba(255, 0, 0, 0.3); }
```

line-height

La propiedad `line-height` permite controlar la altura ocupada por las líneas de texto de un elemento. La diferencia entre la altura de línea y la altura correspondiente al tamaño de la fuente determinará por tanto el interlineado entre líneas.

Propiedad	**line-height**
Valores	normal
Se aplica a	Todos los elementos
Valor inicial	normal
Heredada	Sí
Descripción	Permite establecer la altura de línea de los elementos

Además de todas las unidades de medida y el uso de porcentajes, la propiedad line-height permite indicar un número sin unidades que se interpreta como el múltiplo del tamaño de letra del elemento. Por tanto, estas tres reglas CSS son equivalentes:

```
p { line-height: 1.2; font-size: 1em; }
p { line-height: 1.2em; font-size: 1em; }
p { line-height: 120%; font-size: 1em; }
```

Siempre que se utilice de forma moderada, el interlineado mejora notablemente la legibilidad de un texto.

Veamos una demostración:

See the Pen Propiedad line-height by Pedro Joya (@pedrojoya) on CodePen.

```
<p>Lorem ipsum dolor sit amet, consectetur adipisicing elit. Doloribus ut quae
repellendus perferendis similique, repellat nostrum. Tenetur saepe at distinctio
quas voluptates illum, praesentium tempora! Nemo deleniti vel aliquid quasi!</p>
<p class="separado">Lorem ipsum dolor sit amet, consectetur adipisicing elit.
Aliquid amet molestiae adipisci aperiam alias, fugit molestias nulla sapiente
sunt quam eaque impedit dolores incidunt soluta veritatis dicta perspiciatis modi
odit.</p>
<p class="mas-separado">Lorem ipsum dolor sit amet, consectetur adipisicing elit.
Nisi fugit minima consequatur, repellat fugiat temporibus nemo facere perferendis
molestiae consequuntur ad, necessitatibus suscipit enim blanditiis eos ullam
ratione cupiditate vitae.</p>
<p class="muy-separado">Lorem ipsum dolor sit amet, consectetur adipisicing elit.
Labore natus, unde inventore ipsam facilis sequi perferendis qui consectetur
quisquam consequuntur quibusdam incidunt quae libero, atque maiores praesentium
quia accusamus magnam.</p>
```

```
p.separado {
  line-height: 1.3em;
}
p.mas-separado {
  line-height: 1.6em;
}
p.muy-separado {
  line-height: 2em;
}
```

> Demo de propiedad line-hight

Un aspecto importante que debemos tener en cuenta es que el texto de la línea siempre se centra verticalmente respecto a la altura de línea especificada con `line-height`. Esto hace que una de las técnicas para central verticalmente una única línea de texto dentro de una caja sea simplemente hacer que el valor de `line-height` sea el mismo que el de la altura de la caja. Por ejemplo:

See the Pen Centrado vertical con line-height by Pedro Joya (@pedrojoya) on CodePen.

```
<h1>Texto centrado</h1>
```

```
h1 {
  height: 100px;
  border: 2px solid red;
  text-align: center;
  line-height: 100px;
}
```

> Demo centrado vertical con propiedad line-height

text-align

La propiedad `text-align` permite alinear el texto, dentro de un elemento, a su izquierda o derecha, centrada o justificado (ocupando todo el ancho posible).

Propiedad	**text-align**
Valores	left | right | center | justify | inherit
Se aplica a	Elementos de bloque y celdas de tabla
Valor inicial	left
Heredada	Sí
Descripción	Establece la alineación del contenido del elemento

La propiedad text-align no sólo alinea el texto que contiene un elemento, sino que también alinea todos sus contenidos, como por ejemplo las imágenes.

Muy relacionada con esta propiedad tenemos `text-align-last`, que indica la alineación de la última línea cuando `text-align` tiene el valor `justify`.

Propiedad	text-align-last
Valores	auto | start | end | left | right | center | justify
Se aplica a	Elementos de bloque
Valor inicial	auto
Heredada	Sí
Descripción	Establece la alineación de la última línea de un contenido justificado

Veamos un ejemplo:

See the Pen Propiedad text-align by Pedro Joya (@pedrojoya) on CodePen.

```
<p>Lorem ipsum dolor sit amet, consectetur adipisicing elit. Doloribus ut quae repellendus perferendis similique, repellat nostrum. Tenetur saepe at distinctio quas voluptates illum, praesentium tempora! Nemo deleniti vel aliquid quasi!</p>
<p class="derecha">Lorem ipsum dolor sit amet, consectetur adipisicing elit. Aliquid amet molestiae adipisci aperiam alias, fugit molestias nulla sapiente sunt quam eaque impedit dolores incidunt soluta veritatis dicta perspiciatis modi odit.</p>
<p class="centrado">Lorem ipsum dolor sit amet, consectetur adipisicing elit. Nisi fugit minima consequatur, repellat fugiat temporibus nemo facere perferendis molestiae consequuntur ad, necessitatibus suscipit enim blanditiis eos ullam ratione cupiditate vitae.</p>
<p class="justificado">Lorem ipsum dolor sit amet, consectetur adipisicing elit. Labore natus, unde inventore ipsam facilis sequi perferendis qui consectetur quisquam consequuntur quibusdam incidunt quae libero, atque maiores praesentium quia accusamus magnam.</p>
<p class="justificado finalderecha">Lorem ipsum dolor sit amet, consectetur adipisicing elit. Dolorum inventore fuga excepturi reiciendis unde veritatis. Ipsum, quibusdam itaque assumenda blanditiis placeat, accusantium quis reiciendis odio, facilis asperiores sunt temporibus, quidem?</p>
```

```
p.derecha {
  text-align: right;
}
p.centrado {
  text-align: center;
}
p.justificado {
  text-align: justify;
}
p.finalderecha {
  text-align-last: right;
}
```

> Demo de las propiedades text-align y text-align-last

CSS3 también define la propiedad `text-justify` que permite indicar el método que debe seguirse para justificar el texto cuando `text-align` tiene el valor `justify`. Esta propiedad aún no es soportada por casi ningún navegador.

Propiedad	text-justify
Valores	auto | none | inter-word | inter-character
Se aplica a	Elementos de bloque
Valor inicial	auto
Descripción	Establece el método de justificación a usar cuando `text-align` se ha establecido al valor `justify`

text-indent

En muchas publicaciones impresas suele ser habitual tabular la primera línea de cada párrafo para facilitar su lectura. CSS permite controlar esta tabulación mediante la propiedad text-indent.

Propiedad	text-indent
Valores	unidad de medida | porcentaje | inherit
Se aplica a	Los elementos de bloque y las celdas de tabla
Valor inicial	0
Heredada	Sí
Descripción	Establece una sangría francesa (desde la izquierda a la primera línea) del texto original

Veamos un ejemplo:

See the Pen Propiedad text-indent by Pedro Joya (@pedrojoya) on CodePen.

```
<p>Lorem ipsum dolor sit amet, consectetur adipisicing elit. Aut a molestias
repudiandae maxime eius amet quos repellendus reprehenderit soluta doloribus!
Dicta dolores aliquam ea quas blanditiis unde nam ex necessitatibus.</p>
<p class="identado">Lorem ipsum dolor sit amet, consectetur adipisicing elit.
Aut, repellendus rerum quos ea quasi doloremque qui modi impedit id voluptate
earum atque iure, veritatis distinctio magni iste, doloribus ex praesentium.</p>
```

```
p.identado {
  text-indent: 3em;
}
```

> Demo de la propiedad text-indent

text-decoration

La propiedad `text-decoration` decora el texto con una línea.

Propiedad	text-decoration
Valores	none | (underline | overline | line-through) | inherit

Propiedad	text-decoration
Se aplica a	Todos los elementos
Valor inicial	none
Heredada	**No**
Descripción	Establece la decoración del texto (subrayado, tachado, etc.)

Veamos un ejemplo:

See the Pen Propiedad text-indent by Pedro Joya (@pedrojoya) on CodePen.

```
<p>Soy un texto normal</p>
<p class="subrayado">Soy un texto subrayado</p>
<p class="tachado">Soy un texto tachado</p>
<p class="overline">Texto con una línea por encima</p>
<a href="#" class="sinlinea">Soy un enlace sin subrayar</a>
```

```
p.subrayado {
  text-decoration: underline;
}
p.tachado {
  text-decoration: line-through;
}
p.overline {
  text-decoration: overline;
}
a.sinlinea {
  text-decoration: none;
}
```

> Demo de la propiedad text-decoration

CSS3 ha convertido la propiedad `text-decoration` en una propiedad atajo (*shorthand*) de una serie de propiedades que permiten detallar la decoración que se realiza sobre el texto:

- `text-decoration-color`: Color de la línea de decoración.
- `text-decoration-line`: Posición de la línea de decoración: `underline` (por debajo), `line-through` (atravesando el texto), `overline` (por encima).
- `text-decoration-style`: Estilo de la línea de decoración: `solid` (contínua), `double` (contínua doble), `dotted` (punteada), `dashed` (con guiones), `wavy` (ondulada).
- `text-decoration-skip`: Qué partes del texto NO deben verse afectadas por la línea de decoración.
- `text-underline-position`: Separación de la línea de decoración.
  Actualmente estas propiedades no son soportadas por la mayoría de los navegadores.

text-transform

La propiedad text-transform permite mostrar el texto original transformado en un texto completamente en mayúsculas (`uppercase`), en minúsculas (`lowercase`) o con la primera letra de cada palabra en mayúscula (`capitalize`).

Propiedad	**text-transform**
Valores	capitalize | uppercase | lowercase | none | inherit
Se aplica a	Todos los elementos
Valor inicial	none
Heredada	Sí
Descripción	Transforma el texto original (lo transforma a mayúsculas, a minúsculas, etc.)

Veamos un ejemplo:

See the Pen Propiedad text-transform by Pedro Joya (@pedrojoya) on CodePen.

```
<p>Texto normal</p>
<p class="uppercase">Texto en mayúsculas</p>
<p class="lowercase">Texto en minúsculas</p>
<p class="capitalize">Cada palabra empieza en mayúsculas</p>
```

```
p.uppercase {
  text-transform: uppercase;
}
p.lowercase {
  text-transform: lowercase;
}
p.capitalize {
  text-transform: capitalize;
}
```

> Demo de la propiedad text-transform

direction y unicode-bidi

La propiedad `direction` permite especificar la dirección en la que se debe escribir el texto: `ltr` (de izquierda a derecha, *left to right*) o `rtl` (de derecha a izquierda, *right to left*).

Propiedad	**direction**
Valores	ltr | rtl | inherit
Se aplica a	Todos los elementos
Valor inicial	ltr
Heredada	Sí
Descripción	Especifica la dirección del texto

Esta propiedad se utiliza a menudo en conjunción con la propiedad `unicode-bidi`, que permite que la letras se muestren en "espejo" de derecha a izquierda. Su valor más importante es `bidi-override`. Es una propiedad que no se hereda por defecto.

Veamos un ejemplo:

See the Pen Propiedades direction y unicode-bidi by Pedro Joya (@pedrojoya) on CodePen.

```
<p> Lorem ipsum dolor sit amet, consectetur adipisicing elit. Doloribus ut quae repellendus perferendis similique, repellat nostrum. Tenetur saepe at distinctio quas voluptates illum, praesentium tempora! Nemo deleniti vel aliquid quasi!</p>
<p class="rtl">Lorem ipsum dolor sit amet, consectetur adipisicing elit. Aliquid amet molestiae adipisci aperiam alias, fugit molestias nulla sapiente sunt quam eaque impedit dolores incidunt soluta veritatis dicta perspiciatis modi odit.</p>
<p class="rtl bidi">Lorem ipsum dolor sit amet, consectetur adipisicing elit. Nisi fugit minima consequatur, repellat fugiat temporibus nemo facere perferendis molestiae consequuntur ad, necessitatibus suscipit enim blanditiis eos ullam ratione cupiditate vitae.</p>
```

```
p.rtl {
  direction: rtl;
}
p.bidi {
  unicode-bidi: bidi-override;
}
```

> Demo de las propiedades direction y unicode-bidi

letter-spacing y word-spacing

CSS también permite controlar la separación entre las letras que forman las palabras y la separación entre las palabras que forman los textos.

La propiedad que controla la separación entre letras se llama `letter-spacing`.

Propiedad	**letter-spacing**
Valores	normal | unidad de medida | inherit
Se aplica a	Todos los elementos
Valor inicial	normal
Heredada	Sí
Descripción	Permite establecer el espacio entre las letras que forman las palabras del texto

La separación entre palabras se controla mediante `word-spacing`.

Propiedad	**word-spacing**
Valores	normal | unidad de medida | inherit
Se aplica a	Todos los elementos

Propiedad	word-spacing
Valor inicial	normal
Heredada	Sí
Descripción	Permite establecer el espacio entre las palabras que forman el texto

Veamos un ejemplo:

See the Pen Propiedades letter-spacing y word-spacing by Pedro Joya (@pedrojoya) on CodePen.

```
<div>
  <h1>Lorem ipsum dolor sit amet</h1>
  <p>Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Vestibulum dictum.
Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos
hymenaeos. Etiam pharetra blandit purus.</p>
</div>
<div class="especial">
  <h1>Lorem ipsum dolor sit amet</h1>
  <p>Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Vestibulum dictum.
Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos
hymenaeos. Etiam pharetra blandit purus.</p>
</div>
```

```
.especial h1 {
  letter-spacing: .2em;
}
.especial p {
  word-spacing: .5em;
}
```

> Demo de las propiedades letter-spacing y word-spacing

text-shadow

La propiedad `text-shadow` se usa para aplicar sombras, sombras descendientes, brillo externo y otros efectos de sombra al texto.

Propiedad	text-shadow
Valores	(despl_eje_x despl_eje_y radio_desenfoque color) | none | inherit
Se aplica a	Todos los elementos
Valor inicial	none
Heredada	Sí
Descripción	Permite establecer la sombra del texto

Los dos primeros parámetros, desplazamiento en el eje x y desplazamiento en el eje y, son obligatorios. Si no se especifica color se usa el valor de la propiedad `color` (incluso aunque sea heredada).

Para establecer la sombra hacia la izquierda y/o hacia arriba se usarán valores negativos para el desplazamiento en el eje x y en el eje y respectivamente.

Podemos establecer varias sombras para el texto, separándolas con el carácter `,` (coma).

Usando varias sombras podemos crear efectos muy interesantes, como texto con contorno, texto que resplandece, texto borroso o texto que simula estar en tres dimensiones.

Veamos una demo interactiva:

> [Demo interactiva de text-shadow]

Finalmente veamos varios ejemplos:

See the Pen [Propiedad text-shadow] by Pedro Joya ([@pedrojoya]) on [CodePen].

```
<div class="caja">
  <h2>Texto simple</h2>
</div>
<div class="caja">
  <h2 class="sombra">Con sombra</h2>
</div>
<div class="caja">
  <h2 class="oscuro-sobre-oscuro">Oscuro sobre oscuro</h2>
</div>
<div class="caja">
  <h2 class="claro-sobre-oscuro">Claro sobre oscuro</h2>
</div>
<div class="caja">
  <h2 class="contorno">Con contorno</h2>
</div>
<div class="caja">
  <h2 class="resplandeciente">Resplandeciente</h2>
</div>
<div class="caja">
  <h2 class="mareante">Mareante</h2>
</div>
<div class="caja">
  <h2 class="tres-dimensiones">Pseudo 3D</h2>
</div>
<div class="caja">
  <h2 class="retro">Retro</h2>
</div>
<div class="caja">
  <h2 class="doble">Doble</h2>
</div>
<div class="caja">
  <h2 class="neon">Neón</h2>
</div>
<div class="caja">
  <h2 class="fuego">Fuego</h2>
</div>
<div class="caja">
  <h2 class="grabado">Grabado</h2>
</div>
```

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
body {
  width: 80%;
  margin: 1rem auto;
}
.caja {
  padding: 1rem;
  background-color: #715683;
  text-align: center;
  color: white;
  font-family: Helvetica, sans-serif;
  font-size: 2rem;
}
.sombra {
  text-shadow: 3px 5px 3px #333;
}
/* Para texto oscuro sobre fondo oscuro, usar sombra clara en la parte inferior
*/
.oscuro-sobre-oscuro {
  background: hsl(210, 13%, 60%);
  color: hsl(210, 13%, 30%);
  text-shadow: 0 1px 1px hsla(0, 0%, 100%, .8);
}
/* Para texto claro sobre fondo oscuro, usar sombra oscura en la parte superior
*/
.claro-sobre-oscuro {
  color: hsl(210, 13%, 75%);
  text-shadow: 0 -1px 1px black;
}
.contorno {
  text-shadow: 1px 1px black, -1px -1px black, 1px -1px black, -1px 1px black;
}
.resplandeciente {
  color: #ffc;
  text-shadow: 0 0 .1em, 0 0 .3em;
}
.mareante {
  color: transparent;
  text-shadow: 0 0 .1em white, 0 0 .3em white;
}
.tres-dimensiones {
  text-shadow: 0 1px 0 #CCCCCC, 0 2px 0 #B5B5B5, 0 3px 0 #9B9B9B, 0 4px 0
#898989, 0 5px 0 #7B7B7B, 0 6px 0 #626262, 0 7px 0 rgba(0, 0, 0, 0.5), 0 7px 0
black;
}
.retro {
  text-transform: uppercase;
  letter-spacing: .1rem;
  text-shadow: 1px 1px black, 2px 2px black, 3px 3px black, 4px 4px black, 5px
5px black, 6px 6px black, 7px 7px black, 8px 8px black;
}
.doble {
```

```
  background-color: white;
  color: #323131;
  text-shadow: 4px 3px 0px #fff, 9px 8px 0px rgba(0, 0, 0, 0.15);
}
.neon {
  text-shadow: 0 0 5px #fff, 0 0 10px #fff, 0 0 15px #fff, 0 0 20px #ff2d95, 0 0
30px #ff2d95, 0 0 40px #ff2d95, 0 0 50px #ff2d95, 0 0 75px #ff2d95;
}
.fuego {
  text-shadow: 0px -1px 4px white, 0px -2px 10px yellow, 0px -10px 20px #ff8000,
0px -18px 40px red, 0px 1px 4px white, 0px 2px 10px yellow, 0px 10px 20px
#ff8000, 0px 18px 40px red;
}
.grabado {
  text-transform: uppercase;
  text-shadow: -1px -1px 1px #fff, 1px 1px 1px #000;
  color: #9E8FA8;
  opacity: 0.2;
}
```

> Demo de la propiedad text-shadow

Puede consultar otros tipos de sombras habituales en la siguiente galería de sombras de fuentes artísticas.

Espacios en blanco, saltos de línea, tabuladores y *wrapping*

El aspecto más sorprendente del lenguaje HTML cuando se desarrollan los primeros documentos es el tratamiento especial de los "espacios en blanco" del texto. HTML considera espacio en blanco a los espacios en blanco, los tabuladores, los retornos de carro y el carácter de nueva línea (ENTER o Intro).

El siguiente ejemplo ilustra este comportamiento:

See the Pen Espacios en blanco by Pedro Joya (@pedrojoya) on CodePen.

```
<p>Este primer párrafo no contiene saltos de línea ni otro tipo de espaciado.</p>
<p>Este segundo párrafo sí que contiene saltos
  de
  línea
  y   otro   tipo   de   espaciado.</p>
```

> Demo de espacios en blanco y tabuladores

Los dos párrafos del ejemplo anterior se ven idénticos, aunque el segundo párrafo incluye varios espacios en blanco y está escrito en varias líneas diferentes. La razón de este comportamiento es que HTML ignora todos los espacios en blanco sobrantes, es decir, todos los espacios en blanco que no son el espacio en blanco que separa las palabras.

No obstante, HTML proporciona varias alternativas para poder incluir tantos espacios en blanco y tantas nuevas líneas como sean necesarias dentro del contenido textual de las páginas.

Etiqueta `<br>`

Para incluir una nueva línea en un punto y forzar a que el texto que sigue se muestre en la línea inferior, se utiliza la etiqueta `<br />`. En cierta manera, insertar la etiqueta `<br />` en un determinado punto del texto equivale a presionar la tecla ENTER (o Intro) en ese mismo punto. Se trata de un elemento en línea que no posee etiqueta de cierre.

Utilizando la etiqueta `<br />` se puede rehacer el ejemplo anterior para que respete las líneas que forman el segundo párrafo. Por ejemplo:

See the Pen Etiqueta
by Pedro Joya (@pedrojoya) on CodePen.

```
<p>Este primer párrafo no contiene saltos de línea ni otro tipo de espaciado.</p>
<p>Este segundo párrafo sí que contiene saltos<br />
  de<br />
  línea<br />
  y   otro   tipo   de   espaciado.</p>
```

> Demo de etiqueta `<br>`

Un aspecto importante del elemento `<br />` es que prácticamente no podemos darle un estilo concreto desde CSS, corresponde a un simple salto de línea.

Carácter ` `

Para incluir espacios en blanco adicionales, se debe sustituir cada nuevo espacio en blanco por el texto ` ` (de *non-breaking space*). No debemos olvidar incluir el símbolo `&` del principio y el símbolo `;` del final)

Así, el código HTML del ejemplo anterior se debe rehacer para incluir los espacios en blanco adicionales:

See the Pen Carácter nbsp by Pedro Joya (@pedrojoya) on CodePen.

```
<p>Este primer párrafo no contiene saltos de línea ni otro tipo de espaciado.</p>
<p>Este segundo párrafo sí que contiene saltos<br />
  de<br />
  línea<br />
  y    otro   tipo   de   espaciado.</p>
```

> Demo de carácter ` `

Etiqueta `<wbr>`

Cuando debemos escribir un texto y tenemos disponible un área de visualización con un ancho determinado es muy posible que dicho texto no quepa en una única línea. Se conoce como *line wrap* o *word wrap* a la técnica de continuar el texto en una nueva línea cuando la anterior está completa, de manera que cada línea quepa en el área de visualización, permitiendo que el texto pueda leerse de arriba a abajo si tener que hacer *scroll* horizontal.

El comportamiento por defecto de esta técnica es intentar no romper ninguna palabra por la mitad, sino usar los espacios en blanco y otros caracteres de puntuación para para determinar donde termina una palabra, de manera que si la siguiente palabra del texto no entra en la línea actual, se crea una nueva línea y se escribe en ella dicha palabra.

Sin embargo, cuando tenemos palabras muy largas o largas cadenas de texto sin espacios en blanco, el efecto visual de esta técnica puede no ser el adecuado, ya que puede ocurrir que al no caber la palabra muy larga se cree una nueva línea y la línea anterior quede muy vacía.

Para estos casos HTML proporciona la etiqueta `<wbr />` (de *word break*), que se inserta entre los caracteres de la palabra larga para indicar un posición que puede usarse si es necesario para partir la palabra en dos líneas.

A diferencia de `<br />`, la etiqueta `<wbr />` no obliga a hacer un salto de línea, sólo sugiere una posición idónea para hacerlo si es necesario.

Veamos un ejemplo:

See the Pen Etiqueta wbr y atributo CSS word-wrap by Pedro Joya (@pedrojoya) on CodePen.

```
<p>Este primer párrafo no contiene saltos de línea ni otro tipo de espaciado
esterno<wbr />cleido<wbr />mastoideo</p>
<p>Mirad todos, Esto es una palabra
muuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuy larga</p>
```

```
/* Si una palabra no entra entera incluso si ocupa
   toda la línea, se fuerza a que se divida.
   Si no lo establecemos, se hace scroll */
p {
  word-wrap: break-word;
}
```

> Demo de la etiqueta `<wbr>`

Etiqueta `<pre>`

En ocasiones, es necesario mostrar los espacios en blanco, tabuladores y saltos de línea de un texto tal y como aparecen en él. Se trata de un caso habitual cuando una página web debe mostrar directamente el texto generado por alguna aplicación.

En estos casos, se puede utiliza la etiqueta `<pre>`, que muestra el texto tal y como se ha escrito, respetando todos los espacios en blanco y todas las nuevas líneas. Se trata de un elemento en bloque. Veamos un ejemplo:

See the Pen Etiqueta pre by Pedro Joya (@pedrojoya) on CodePen.

```
<pre>
        La etiqueta pre
   respeta los espacios en blanco,                    no hace word wrapping
      y
 muestra el texto
     tal y como
   está escrito
</pre>
<p>
```

```
        La etiqueta pre
   respeta los espacios en blanco,                    no hace word wrapping
      y
 muestra el texto
     tal y como
   está escrito
</p>
```

> Demo de la etiqueta `<pre>`

El primer texto se ve en pantalla tal y como se ha escrito, respetando todos los espacios en blanco y todas las nuevas líneas. El segundo texto se ve como un párrafo normal, ya que HTML ha eliminado todos los espacios en blanco sobrantes.

Los elementos `<pre>` son especiales, ya que los navegadores les aplican las siguientes reglas:

- Mantienen todos los espacios en blanco (tabuladores, espacios y nuevas líneas)
- Por defecto, el navegador muestra el texto con un tipo de letra especial, denominado "de ancho fijo", ya que todas sus letras son de la misma anchura
- No se ajusta la longitud de las líneas, es decir, se hace *word wrapping*, por lo que las líneas largas producen un scroll horizontal en la ventana del navegador.

Propiedades CSS relacionadas con espacios en blanco, tabulaciones, saltos de línea y *wrapping*

white-space

CSS permite controlar el tratamiento de los espacios en blanco de los textos mediante la propiedad `white-space`.

Propiedad	**white-space**
Valores	normal | pre | nowrap | pre-wrap | pre-line | inherit
Se aplica a	Todos los elementos
Valor inicial	normal
Heredada	Sí
Descripción	Establece el tratamiento de los espacios en blanco del texto

El significado de cada uno de los valores es el siguiente:

- `normal`: comportamiento por defecto de HTML.
- `pre`: se respetan los espacios en blanco y las nuevas líneas (exactamente igual que la etiqueta `<pre>`). Si la línea es muy larga, se sale del espacio asignado para ese contenido.
- `nowrap`: elimina los espacios en blanco y las nuevas líneas. Si la línea es muy larga, se sale del espacio asignado para ese contenido.
- `pre-wrap`: se respetan los espacios en blanco y las nuevas líneas, pero ajustando cada línea al espacio asignado para ese contenido.

- `pre-line`: elimina los espacios en blanco y respeta las nuevas líneas, pero ajustando cada línea al espacio asignado para ese contenido.
  En la siguiente tabla se resumen las características de cada valor:

Valor	Respeta espacios en blanco	Respeta saltos de línea	Ajusta las líneas
normal	no	no	si
pre	si	si	no
nowrap	no	no	no
pre-wrap	si	si	si
pre-line	no	si	si

Veamos un ejemplo:

See the Pen [Propiedad white-space](#) by Pedro Joya (@pedrojoya) on CodePen.

```
<p>Lorem ipsum dolor sit amet, consectetur adipisicing elit.
  Incidunt dolorem ipsam accusamus deleniti totam deserunt a, illo harum cumque
ab officiis voluptate molestiae eveniet soluta quisquam itaque, odit! Voluptatem,
incidunt.</p>
<p class="pre">Lorem ipsum dolor sit amet, consectetur adipisicing elit.
  Possimus officiis, aliquam non dolorum sed exercitationem ipsam rem asperiores
et voluptates similique, aut sapiente harum accusantium esse neque voluptatibus
praesentium rerum.</p>
<p class="pre-wrap">Lorem ipsum dolor sit amet, consectetur adipisicing elit.
  Quae labore voluptatem, culpa, velit doloribus corrupti suscipit inventore
temporibus sequi obcaecati cupiditate, itaque minima! Fuga et eum fugiat deleniti
officiis nobis.</p>
<p class="nowrap">Lorem ipsum dolor sit amet, consectetur adipisicing elit.
  Accusantium quidem voluptates perspiciatis cupiditate maxime, illo quisquam
neque modi commodi doloremque earum voluptatum libero omnis excepturi fugiat
quas, aspernatur sint. Voluptatem?</p>
<p class="pre-line">Lorem ipsum dolor sit amet, consectetur adipisicing elit.
  Repellendus quae, sint dolorem nemo modi vel incidunt eveniet dolorum,
accusamus repudiandae beatae voluptatem in aliquid dolore magnam sit quaerat.
Laboriosam, non.</p>
```

```
p {
  width: 300px;
  border: solid 2px black;
}
.pre {
  white-space: pre;
}
.pre-wrap {
  white-space: pre-wrap;
}
.nowrap {
  white-space: nowrap;
}
.pre-line {
```

```
  white-space: pre-line;
}
```

> Demo de la propiedad white-space

tab-size

La propiedad `tab-size` permite especificar el espacio que ocupa el carácter tabulador (unicode U+0020). Sólo tiene efecto cuando se debe mostrar en un elemento cuya propiedad `white-space` tiene el valor `pre` o `pre-wrap`.
El valor recibido corresponde al número de caracteres espacio en blanco con lo que va a representarse el tabulador.

Propiedad	tab-size
Valores	
Se aplica a	Elementos en bloque
Valor inicial	8
Heredada	Sí
Descripción	Establece el número de caracteres con tratamiento de los espacios en blanco del texto

El valor por defecto de la propiedad es 8 espacios en blanco.

Veamos un ejemplo:

See the Pen Propiedad tab-size by Pedro Joya (@pedrojoya) on CodePen.

```
<p>while (x > 0) {&NewLine;&Tab;x = x - 1;}</p>
<p class="pre">while (x > 0) {&NewLine;&Tab;x = x - 1;&NewLine;}</p>
<p class="pre-wrap">while (x > 0) {&NewLine;&Tab;x = x - 1;&NewLine;}</p>
<p class="nowrap">while (x > 0) {&NewLine;&Tab;x = x - 1;&NewLine;}</p>
<p class="pre-line">while (x > 0) {&NewLine;&Tab;x = x - 1;&NewLine;}</p>
```

```
p {
  tab-size: 4;
}
.pre {
  white-space: pre;
}
.pre-wrap {
  white-space: pre-wrap;
}
.nowrap {
  white-space: nowrap;
}
.pre-line {
  white-space: pre-line;
}
```

> Demo de la propiedad tab-size

word-wrap, overflow-wrap y word-break

Las propiedades `word-wrap` y `overflow-wrap` son en realidad dos nombres distintos para la misma propiedad.Se usa para especificar si para prevenir el *overflow* (cuando una palabra no cabe en su caja) se desea forzar la ruptura en un punto arbitrario de palabras que no se pueden romper naturalmente.

Estas propiedades solo tienen efecto cuando la propiedad `white-space` tiene un valor que permite el *wrapping*.

Propiedad	**word-wrap / overflow-wrap**
Valores	normal | break-word
Se aplica a	Elementos en bloque
Valor inicial	normal
Heredada	Sí
Descripción	Establece si se debe forzar la ruptura de palabras en puntos arbitrarios

La propiedad `word-break` permite, por su parte, establecer la estrategia a seguir para la ruptura de palabras por el *wrapping*.

Propiedad	**word-break**
Valores	normal | break-all | keep-all
Se aplica a	Todos los elementos
Valor inicial	normal
Descripción	Establece la estrategia de ruptura de palabras por el wrapping

Los valores posibles son:

- `normal`: Usa las reglas por defecto para la ruptura de palabras.
- `break-all`: Se puede romper la palabra en cualquier letra.
- `keep-all`: En los idiomas chino, japonés y coreano no se pueden romper las palabras. Para otros idiomas, el valor es igual que `normal`.
  El empleo de esta propiedad puede producir rupturas en posiciones no adecuadas.

hyphens

La propiedad `hyphens` se usa para que al hacer wrapping se rompa automáticamente la palabra en una posición adecuada, atendiendo a las reglas del idioma correspondiente.

Propiedad	**hyphens**
Valores	none

Propiedad	hyphens
Se aplica a	Todos los elementos
Valor inicial	normal
Descripción	Establece la estrategia de ruptura de palabras por el wrapping

Los valores posibles son:

- `none`: La palabras nunca son rotas, incluso aunque contengan caracteres que sugieran que es esa es una posición adecuada para partir la palabra.
- `manual`: Las palabras sólo pueden romperse (y añadir el guión) en las posiciones marcadas con caracteres especiales de posibilidad de ruptura:
  - `U+2010 (hyphen)`: que es visible con un `-`.
  - `U+00AD (shy, soft hyphen)`: que es invisible. En HTML se usa el carácter ``
- `auto`: Las palabras se podrán romper sólo en posiciones adecuadas, atendiendo a las reglas del idioma correspondiente, que debe ser soportado por el navegador.
  Para que funcione correctamente es necesarios que especifiquemos el idioma en la página mediante el atributo `lang` de l etiqueta `<html>`. Actualmente, esta propiedad NO es soportada por Chrome, aunque sí por Firefox y por Safari (usando el prefijo `-webkit-`).

Veamos un ejemplo:

See the Pen [Propiedades word-break y hyphen](#) by Pedro Joya (@pedrojoya) on CodePen.

```
<p>En un lugar de la Mancha, de cuyo nombre no quiero acordarme, no ha mucho
tiempo que vivía un hidalgo de los de lanza en astillero, adarga antigua, rocín
flaco y galgo corredor. No funciona en Chrome.</p>
<p class="guiones">En un lugar de la Mancha, de cuyo nombre no quiero acordarme,
no ha mucho tiempo que vivía un hidalgo de los de lanza en astillero, adarga
antigua, rocín flaco y galgo corredor. No funciona en Chrome.</p>
```

```
p {
  width: 50%;
  border: solid 2px black;
}
.guiones {
  -webkit-hyphens: auto;
  hyphens: auto;
}
```

> Demo de las propiedades word-break y hyphens

text-overflow

La propiedad `text-overflow` especifica cómo debe tratarse el texto cuando es recortado debido a que es más largo que el bloque que lo contiene, de manera que se muestre una elipsis (`...`). Sólo funciona cuando el texto tiene una única línea.

Propiedad	text-overflow
Valores	clip | ellipsis
Se aplica a	Todos los elementos
Valor inicial	clip
Descripción	Establece cómo se recorta el texto cuando no cabe en la línea

El valor `clip` simplemente recorta el texto, mientras que el valor `ellipsis` añade puntos suspensivos.

Veamos un ejemplo:

See the Pen [Propiedad text-overflow](#) by Pedro Joya (@pedrojoya) on CodePen.

```
<p>Lorem ipsum dolor sit amet, consectetur adipisicing elit. Illo voluptate odio autem, totam quas. Atque, autem quaerat labore, rem sapiente corporis reiciendis maiores modi aliquid repudiandae cumque. Eum, cum, quod.</p>
<p class="elipsis">Lorem ipsum dolor sit amet, consectetur adipisicing elit. Unde dolore, quasi, recusandae cumque harum aspernatur accusamus reiciendis asperiores reprehenderit voluptatum quas aliquam assumenda eveniet quod, dolor delectus, illum odit. Odit?</p>
```

```
p {
  width: 20em;
    overflow: hidden;
    white-space: nowrap;
}
p.elipsis {
    text-overflow: ellipsis;
}
```

> Demo de la propiedad text-overflow

Pseudoelementos :first-letter y :first-line

El pseudoelemento `:first-letter` permite seleccionar la primera letra del elemento para aplicarle el estilo deseado. El pseudoelemento `:fist-line` permite selecciona la primera línea de texto del elemento.

Veamos un ejemplo:

See the Pen [Pseudoelementos :first-letter y first-line](#) by Pedro Joya (@pedrojoya) on CodePen.

```
<p>En un lugar de la Mancha, de cuyo nombre no quiero acordarme, no ha mucho tiempo que vivía un hidalgo de los de lanza en astillero, adarga antigua, rocín flaco y galgo corredor.</p>
<p>Una olla de algo más vaca que carnero, salpicón las más noches, duelos y quebrantos los sábados, lentejas los viernes, algún palomino de añadidura los domingos, consumían las tres partes de su hacienda.</p>
```

```
p {
  width: 50%;
}
p:first-letter {
  font-size: 2em;
}
p:first-line {
  font-weight: bold;
}
```

> Demo de los pseudoelementos `:first-letter` y `:first-line`